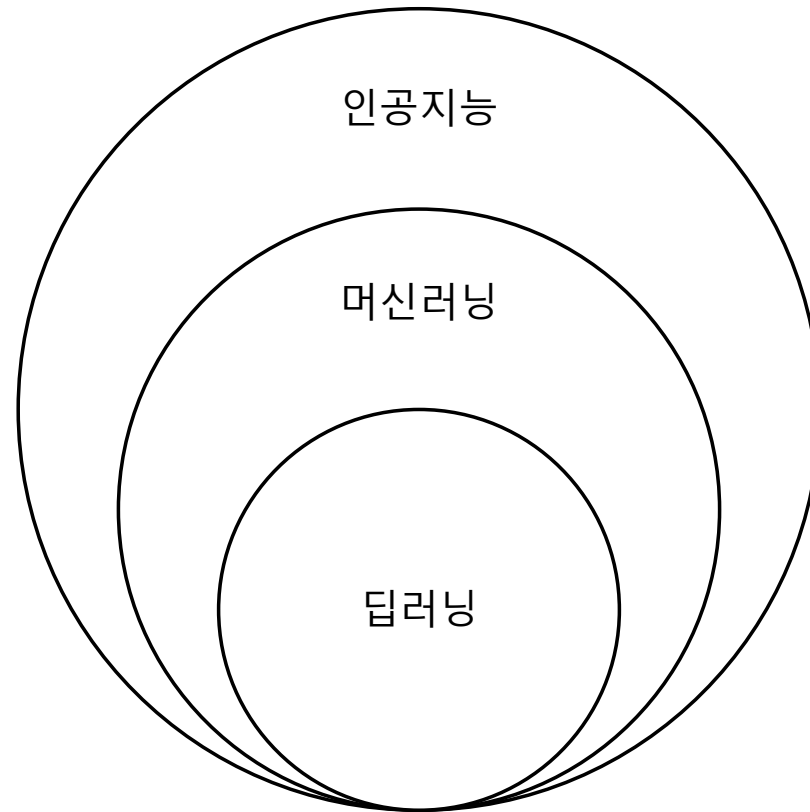


딥 러닝을 이용한 문제 해결 시 고려사항

인공지능과 딥러닝



딥 러닝을 이용한 문제 해결 시 고려사항

1. 문제 정의
2. 데이터셋 정의
3. 모델 선택
4. 하이퍼 파라미터 선택
5. 데이터 전처리
6. 모델 훈련 및 평가
7. 배포 환경에 대한 모델 최적화
8. 배포 후 모델 평가
9. 유지보수

문제 정의: 어떤 문제를 어떤 방식으로 해결할 것인가?

- 수소 탱크의 크랙을 검사하기 위한 기포 검사기의 경우
 - 문제
 - 수소 탱크에 크랙이 있는지 검사해야 함.
 - 검사 방법
 - 기포가 발생할 수 있는 액체를 뿌린 후 기포가 발생하는지 확인
 - 기포의 개수와 영역을 확인
 - 해결 방법
 - 액체를 뿌린 영역을 카메라로 촬영
 - 몇 개의 기포가 있고 각 기포의 영역이 얼마나 되는지 딥 러닝 모델로 검출
 - 이 때 사용할 수 있는 딥 러닝 모델: Instance Segmentation 모델
 - 이미지 분류 모델로는 해결이 가능할 것인가?
 - 기포 유/무를 분류할 수는 있겠으나 기포 개수와 영역을 검출할 수는 없음
 - 객체 인식 모델로는 해결이 가능할 것인가?
 - 기포가 몇 개 있는지는 검출할 수 있겠으나 비정형인 기포의 영역을 검출할 수는 없음

데이터셋 정의:

입력 데이터와 출력 데이터는 어떤 형식이 될 것인가?

- 입력 데이터
 - 카메라로 취득하는 기포 촬영 영상
- 출력 데이터
 - 개별 기포의 영역 정보
- 출력 데이터의 후처리
 - 개별 기포의 영역과 전체 기포 수량, 전체 기포 영역을 계산

데이터셋 정의:

데이터셋은 어떻게 모을 것인가?

- 사용 가능한 공개 데이터셋이 있는 경우
 - 해당 공개 데이터셋을 활용하여 훈련 수행
 - 그러나 기업이 사업을 목적으로 딥 러닝 모델을 훈련하는 경우, 이를 위한 데이터셋은 대부분 공개되지 않음
 - 데이터를 수집하는 것도, annotation하는 것도 모두 돈
- 사용 가능한 공개 데이터셋이 없는 경우
 - 실제 배포 환경에서 이미지를 취득한 후 annotation 수행
 - labelimg, labelme, CVAT 등 공개된 프로그램을 사용하거나 특수한 경우 직접 annotation 프로그램 제작

데이터셋 정의:

직접 데이터셋을 모으는 경우

- 모델 성능의 개선을 위해 유의미한 데이터를 모았는가?
 - 불량 검출을 위해 불량 사진을 찍어야 하지만, 대부분의 경우 양품이 더 많으므로 양품의 사진이 압도적으로 많음
- 실제 배포 환경 또는 그와 유사한 환경에서 수집했는가?
 - 차량의 룸미러 높이에서 전방을 바라보는 자율 주행을 위한 데이터셋을 수집하는데, 차량의 후드에서 전방을 바라보는 데이터를 수집했다면?
- 데이터의 양은 충분한가?
 - 데이터의 양이 너무 적으면 모델이 훈련 데이터에 과적합되는 경우 검증 데이터로 이를 확인하지 못할 수 있음

데이터셋 정의:

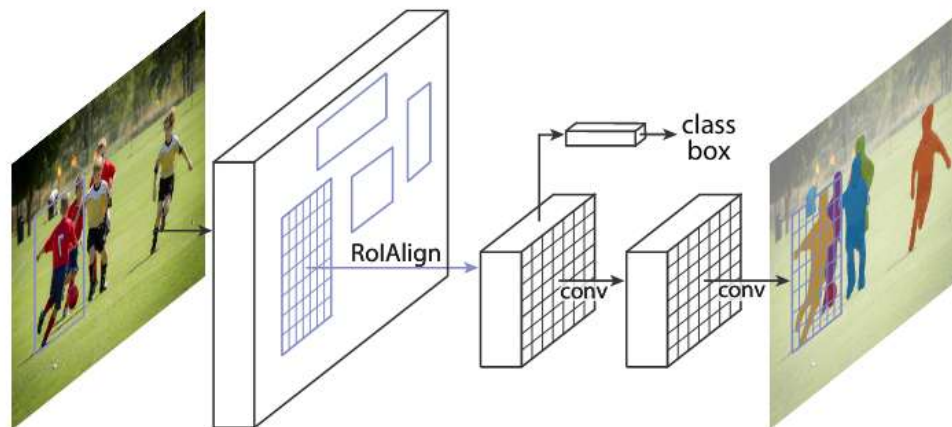
직접 데이터셋을 모으는 경우

- 데이터의 종류는 얼마나 다양한가?
 - 자율주행 개발 시 고속도로, 시내, 톨게이트, 터널 등 다양한 환경에서 데이터를 수집했는가?
 - 불량 검사 개발 시, 다양한 유형의 불량 데이터가 훈련 데이터셋에 있는가?
- Annotation 기준은 오해의 여지가 없는가?
 - 차량을 검출한다면 차량에 box를 그리는 기준은? 차량에 딱 맞게 그릴 때 트럭에 실린 짐은?
- 모델이 출력하고자 하는 것을 정확하게 출력할 수 있게 Annotation 되었는가?
 - 객체의 박스 정보를 통해 안전과 관련한 정보를 계산하는 경우, 박스는 객체에 딱 맞게 준비되어야 함

모델의 선택:

모델은 어떻게 훈련할 것인가?

- 전반적인 훈련 과정 및 배포 과정이 이미 정립되어 있는 경우
 - 그 과정을 따라서 프로젝트 수행
- Framework 선정에 제약이 없는 경우
 - 편한 것 사용
 - 예시에서는 PyTorch MMDetection을 사용하여 Mask R-CNN을 훈련함



모델의 선택:

모델의 연산량에 제약이 있는가?

- 사용 가능한 연산량에 제한이 있고 추론 속도도 빨라야 하는 경우는 모델의 성능을 낮춰야 할 수 있음
- 고객사에서 사용하기를 원하는 SoC가 Nvidia의 Jetson인 경우
 - Nvidia의 Jetson은 ARM CPU와 GPU가 메모리를 공유하고, GPU가 CUDA 연산을 지원하므로 연산의 종류를 고려하지 않고 연산 속도만 고려하면 됨
- 고객사에서 사용하기를 원하는 SoC가 다른 H/W인 경우
 - 일반적으로 딥 러닝 가속기는 지원하는 연산이 제한적이며, 그 외 다른 연산을 사용하는 경우 가속기 – ARM CPU – 가속기 순서로 연산을 수행하게 됨. 이 과정에서 메모리 이동으로 인해 연산시간에 지연이 생기게 됨
 - 모델 내 특정 연산을 다른 연산으로 바꾸거나 지연 시간을 보상하기 위해 모델을 더 가볍게 하는 등의 대안을 고려해야 함

하이퍼 파라미터의 선택

- 하이퍼 파라미터의 선택 또는 다양한 하이퍼 파라미터를 적용한 실험
 - 학습률, 이포크, 배치 크기, 최적화 함수 등을 다양하게 훈련하며 최적의 성능을 가지는 모델 확보
- 모델에 따라, 데이터셋에 따라, 문제에 따라 다르며 정답이 없음.
- GPU에 여유가 있는 경우 다양한 조합으로 훈련하면서 최적의 조합을 찾는 실험을 수행할 수 있음
 - 실험할 때 실험군과 대조군에 대한 차이를 명확하게 실험해야만 함

데이터 전처리:

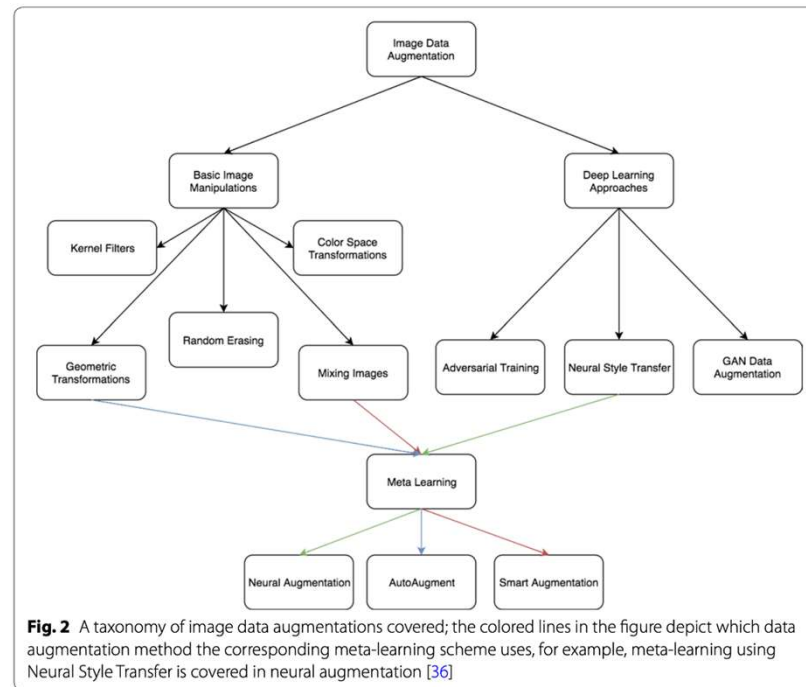
데이터 정규화

- 이미지 정규화
 - 일반적인 이미지 데이터 형식: $[0, 255]$ 의 8bit unsigned int
 - 신경망에 입력하기 위한 이미지 데이터 정규화
 - $[0, 1]$ fp32
 - 신경망의 활성화함수로 ReLU를 사용하는 경우 $[0, 1]$ 정규화를 사용하는 것이 권장됨
 - $[-1, 1]$ fp32
 - 훈련 DB의 평균과 분산을 이용한 $[0, 1]$ 정규분포 변환
 - 일반적으로 ImageNet train DB의 평균/분산을 이용하여 정규분포 변환을 수행함
- 이미지가 아닌 데이터 정규화
 - 훈련 DB의 최대/최소를 이용한 $[0, 1]$ 변환 등
 - 센서 데이터를 딥 러닝 모델의 입력으로 사용하기 위한 다양한 정규화 방법 확인 필요

데이터 전처리:

데이터 증강

- 데이터 증강에 관한 다양한 논문을 적용



Shorten and Khoshgoftaar, A survey on Image Data Augmentation for Deep Learning, Journal of Big Data, 2019

모델 훈련 및 평가:

훈련 시 고려사항

- 준비한 데이터셋을 훈련 데이터와 검증 데이터, 테스트 데이터로 분배할 것인지, 분배한다면 그 비율은 어떻게 할 것인지
 - 이 때, 검증과 테스트 용으로 나눈 데이터는 절대 훈련에 사용하면 안됨
- 과적합을 방지하기 위한 방법은 무엇인지
 - 검증 데이터로 추론한 정확도가 이전 이포크보다 낮아지면 훈련을 중단할 것인지?
 - 과연 몇 이포크동안 낮아질 때 훈련을 중단할 것인지?
 - 정답이 없으며 훈련 데이터셋의 양에 따라 경험적으로 실험

모델 훈련 및 평가:

평가 시 고려사항

- 검증 데이터와 테스트 데이터는 실제 배포 환경과 얼마나 유사한지
- 평가 시 모델의 성능을 어떤 방식으로 수치화 할 것인지
 - mAP@0.5
 - mAP@0.5:0.95
 - Recall
 - F1 score
- 평가를 통해 성능을 수치화할 때, 수치를 신뢰할 수 있을 정도로 평가 데이터는 충분히 있는지
 - 극단적으로 평가 데이터가 이미지 100장일 때, 그 성능 수치를 신뢰할 수 있을지

배포 환경에 대한 모델의 최적화:

Pruning과 Quantization

- Pruning (가지치기)
 - 훈련이 완료된 모델의 가중치 중 연산 결과에 영향을 주지 않는 가중치를 0으로 하여 가속기에서의 연산 시간을 줄이는 것
- Quantization (양자화)
 - 일반적으로 딥 러닝 모델의 훈련과 추론은 FP32 정밀도를 사용
 - FP16, BF16, FP8, INT8, INT4 등 다양한 연산 정밀도로 양자화를 수행할 수 있음
 - SoC가 지원하는 양자화를 확인하고, 아래 두 가지 중 한 가지 방법으로 양자화를 수행할 수 있음
 - Quantization Aware Training (QAT)
 - 훈련 과정에서 quantized precision을 가정, 훈련하며 실시간으로 quantization을 수행
 - Post Training Quantization (PTQ)
 - 훈련이 완료된 모델을 quantization

배포 후 모델 평가:

배포를 가정한 테스트 환경에서 최종 평가 수행

- 테스트 데이터에 대한 검증 수행
- 기존에 배포가 있었던 경우, 모델의 성능이 잘 나오지 않았던 데이터에 대한 테스트 수행
 - 톨게이트 환경에서의 차선 인식 검증
 - 안개 짙은 산골 국도에서 차선 인식 검증
- 모델의 성능을 수치화한 보고서 작성
 - 다양한 조건에서 모델의 성능이 어떻게 되는지 보여줄 수 있는 보고서
 - 배포 환경, 모델의 성능을 표현할 수 있는 수치 등에 대해 이해하고 있어야 작성할 수 있음

유지보수:

배포 후 이슈 발생 시 대응 방안에 대한 고려

- 이슈가 발생한 데이터를 수집하여 훈련 데이터에 추가
 - 가장 쉽고 확실하지만 시간과 돈이 필요한 해결방안
- 이슈를 해결할 수 있도록 배포한 모델의 추론 설정을 수정
 - 불량을 검출하지 못하는 경우가 잦으면 불량 여부를 판단하는 기준을 낮추는 방식

딥 러닝을 이용한 문제 해결 시 고려사항

1. 문제 정의
2. 데이터셋 정의
3. 모델 선택
4. 하이퍼 파라미터 선택
5. 데이터 전처리
6. 모델 훈련 및 평가
7. 배포 환경에 대한 모델 최적화
8. 배포 후 모델 평가
9. 유지보수

문서화

- 업무를 진행하거나 연구를 수행함에 있어서 **가장 중요한 사항**
 - 연구를 진행하는 모든 과정에서 원인, 과정, 결과 분석에 대해 문서를 작성하는 습관은 **매우 중요함**
- 문서화가 잘 되어있지 않다면 아래의 상황에 직면할 수 있음

■ 차장님,
시소프트웨어 담당자들의 연이은 퇴사로 "소프트웨어 조작/운영" 관련 도움을 받기 힘든 상황입니다.
일정 지연 최소화를 위해 "■ A/S"를 긴급히 요청 드립니다.

바쁘시겠지만 빠른 조치 부탁드립니다.

- 문서화가 잘 되어있지 않다면 의지와 무관하게 하던 일을 계속 하게 될 수 있음
 - 일을 혼자 다 할 것이 아니라면, 문서화는 필수

Q&A